



QUICK REFERENCE

MCP Server Setup

Model Context Protocol · Local Development · macOS

TASK	COMMAND / ACTION
1 Check Python version	<code>python3 --version</code>
2 Install uv	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>
3 Find uv path	<code>which uv</code>
4 Create project	<code>mkdir my-server && cd my-server && uv init</code>
5 Install MCP SDK	<code>uv add mcp</code>
6 Test server	<code>uv run server.py</code>
7 Edit config	<code>nano ~/Library/Application\ Support/Claude/claude_desktop_config.json</code>

8

View logs

```
cat ~/Library/Logs/Claude/mcp-server-*.log
```

9

Get project path

```
pwd (inside project folder)
```

10

Restart Claude Desktop

```
Cmd+Q → reopen app
```

MCP · CLAUDE DESKTOP · macOS

docs.anthropic.com

February 25, 2026

From Zero to MCP

A Real-World Guide to Building Your First Model Context Protocol Server

**Sakti Bagchi**

6 min read

[Download article](#) ▾**Contents** 5

1 A Real-World Guide to Building Your First Model Context Protocol Server

- What Exactly Is an MCP Server?
- 1. Python — and the Right Version
- 2. uv — The Python Package Manager That Actually Works
- 3. Claude Desktop

A Real-World Guide to Building Your First Model Context Protocol Server

Including the mistakes nobody warns you about

In my last article, I told you the story of a brilliant detective locked inside a sealed library — brilliant, but cut off from the world. That was AI without MCP.

Then I built the door myself.

I created my own MCP server — a little Python tool I called social-wand, that takes a raw idea, reads an article URL, and crafts polished social media posts for LinkedIn, Twitter, Facebook, and WhatsApp in one shot. Simple concept. Powerful in practice.

What nobody told me was how many walls I'd hit along the way. Python version conflicts. The Chrome extension not recognising my server. Config file paths that silently refused to work. The kind of friction that doesn't show up in tutorials.

This article is the guide I wish I had. Step by step, mistake by mistake, solution by solution.

What Exactly Is an MCP Server?

Before we write a single line of code, let's make sure we're talking about the same thing.

An MCP (Model Context Protocol) server is a small program that sits between Claude (or any AI model) and the outside world. It exposes tools — functions the AI can call to do real things. Read a file. Fetch a URL. Post to an API. Send a message.

Think of it like a power strip. Claude is the appliance. Your tools are the sockets. MCP is the strip that makes everything plug in the same way.

Without MCP, Claude can only think. With MCP, Claude can act.

What You Need Before You Start

Let's get the toolbox ready. Here's what you'll need on your Mac:

1. Python — and the Right Version

This was my first wall. Python version matters more than you think.

MCP SDK requires Python 3.10 or higher. Check yours first:

```
1 python3 - version
```

If you see 3.8 or 3.9, upgrade. The SDK will install but silently fail at runtime — one of those bugs that costs you two hours of debugging. Install Python 3.11 from python.org or via Homebrew:

```
1 brew install python@3.11
```

2. uv — The Python Package Manager That Actually Works

Forget pip for this project. The MCP community runs on uv — a fast, dependency-aware Python package manager. Claude Desktop also expects uv by default when running Python MCP servers.

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

After installing, verify it's accessible:

```
1 which uv
```

It should return something like `/Users/yourname/.local/bin/uv`. Remember this path — you'll need the full path later in the Claude Desktop config. If you type just 'uv' in the config, Claude Desktop won't find it.

3. Claude Desktop

Download Claude Desktop from claude.ai/download. This is the Mac application — not the web browser version. The browser version cannot connect to local MCP servers. This distinction tripped me up when I first tried using the Claude in Chrome extension, expecting it to call my local server. It can't. Claude Desktop is the only client that supports local MCP servers.

Step 1 — Create Your Project Folder

Open Terminal and create a clean home for your server:

```
1 mkdir ~/my-mcp-server
2 cd ~/my-mcp-server
3 uv init
```

uv init creates a `pyproject.toml` file — the project's identity card. It tells uv which Python version to use and which packages to install.

Step 2 — Install the MCP SDK

```
1 uv add mcp
```

This installs the `mcp` package and pins it in your `pyproject.toml`. You'll also see a `.venv` folder appear — uv creates an isolated virtual environment automatically. No need to activate it manually. `uv run` handles that for you.

Step 3 — Write Your Server

Create a file called `server.py`. This is where the magic lives.

Here is a minimal but complete MCP server that exposes one tool — a `hello_world` function. We'll build on this pattern for anything more complex:

```
1 # server.py
2 from mcp.server.fastmcp import FastMCP
3 mcp = FastMCP("my-server")
```

```
4 @mcp.tool()
5 def hello_world(name: str) → str:
6     """Say hello to someone."""
7     return f"Hello, {name}! Your MCP server is working."
8 if __name__ == "__main__":
9     mcp.run()
```

Let's decode what's happening here:

- FastMCP is the high-level wrapper that handles all the protocol wiring for you.
- `@mcp.tool()` is the decorator that registers your function as a tool Claude can call.
- The docstring (the triple-quoted string) becomes the tool's description — Claude reads this to understand when and how to use your tool.
- `mcp.run()` starts the server using stdio transport — the standard way Claude Desktop communicates with local servers.

Step 4 — Test Your Server in Terminal First

Before touching Claude Desktop, always verify your server runs cleanly:

```
1 uv run server.py
```

You should see no errors. The process starts and waits — that's correct behaviour. It's listening for MCP messages. Press Ctrl+C to stop it.

Pro Tip: If you see `ModuleNotFoundError` or `ImportError`, your Python version is incompatible or the SDK didn't install correctly. Run `uv add mcp` again and check your Python version with `uv python list`.

Step 5 — Connect to Claude Desktop

This is the step where most people get stuck. The config file is simple — but the details matter enormously.

Find Your Config File

Open Terminal and run:

```
1 nano ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

Write the Config

Here is the exact pattern. Replace the paths with your own:

```
1 {
2   "mcpServers": {
3     "my-mcp-server": {
4       "command": "/Users/yourname/.local/bin/uv",
5       "args": [
6         "- directory",
7         "/Users/yourname/my-mcp-server",
8         "run",
9         "server.py"
10      ]
11    }
12  }
13 }
```

The Three Rules Nobody Mentions

1. **Always use full absolute paths.** Never write just 'uv' or './server.py'. Claude Desktop runs in a restricted environment that doesn't inherit your shell's PATH. I spent an afternoon on this. The full path to uv (from 'which uv') and the full path to your project folder are non-negotiable.
2. **Restart Claude Desktop completely after every config change.** Cmd+Q, not just closing the window. The MCP server is loaded at startup. If you change the config and just switch windows, nothing updates.
3. **Valid JSON is sacred.** One missing comma, one extra bracket — and the entire config silently fails. Use a JSON validator like jsonlint.com before saving.

Step 6 — Verify the Connection

After restarting Claude Desktop, look near the chat input box. You should see a small 🛠️ hammer icon. Click it. Your server's tools should appear in a dropdown list.

If the hammer icon doesn't appear, something went wrong. Check the logs:

```
1 ls ~/Library/Logs/Claude/  
2 cat ~/Library/Logs/Claude/mcp-server-*.log
```

These logs are your best friend when debugging. They'll show exactly what went wrong — wrong path, missing module, syntax error in server.py. Read them carefully before asking the internet.

The Walls I Hit — And How to Get Past Them

Consider this your field guide to MCP server debugging.

Wall 1: Python version incompatibility

Symptom: Server installs fine but crashes on startup with cryptic errors.

Fix: Run 'python3 — version'. If below 3.10, upgrade. Then run 'uv add mcp' again in your project folder.

Wall 2: Claude Desktop can't find uv

Symptom: Hammer icon appears but tools show 'server failed to start'.

Fix: Replace 'uv' in your config with the full path from 'which uv'. Mine was /Users/saktibagchi/.local/bin/uv.

Wall 3: Chrome extension can't call local MCP servers

Symptom: You try using MCP from Claude's web interface or Chrome extension — nothing happens.

Fix: This is by design. The Claude in Chrome extension handles browser automation only. Local MCP servers only work with Claude Desktop. You can't route around this — use the right tool for the job.

Wall 4: Config file has a JSON error

Symptom: No hammer icon. No errors anywhere visible.

Fix: Open claude_desktop_config.json and paste the contents into jsonlint.com. Invalid JSON fails silently — Claude Desktop simply ignores the config.

Wall 5: Relative paths in the config

Symptom: Works in terminal, fails in Claude Desktop.

Fix: Always use absolute paths starting with /Users/yourname/... in the config file. In terminal, run 'pwd' inside your project folder to get the exact path.

Going Further — My social-wand Example

Once I had the basic server working, I built social-wand — a tool that takes a raw idea and an article URL, and generates platform-specific social media posts in one call. The tool is called `craft_all_platforms`. Here's the stripped-down version of how that tool is defined:

```
1 @mcp.tool()
2 def craft_all_platforms(raw_idea: str, article_url: str = "") → str:
3     """
4     Craft social media posts for LinkedIn, Twitter, Facebook, WhatsApp.
5     raw_idea: Your content idea or topic
6     article_url: Optional URL to the article being promoted
7     """
8     # ... your logic here
```

The docstring does the heavy lifting. Claude reads it, understands the parameters, and knows exactly when to invoke this tool. Write clear docstrings — they are not documentation, they are instructions to the AI.

The Bigger Picture

When I finally saw the 🛠️ hammer icon appear in Claude Desktop with my social-wand tools listed inside it — something clicked that no amount of reading about MCP had given me.

The detective had walked out of the library.

MCP is not just a developer curiosity. It is the plumbing that turns an AI model into an AI agent. Every tool you expose through an MCP server is a new capability Claude can use on your behalf — reading your files, calling your APIs, automating your workflows.

And you built the gateway.

Start simple. One tool. Make it run. Then add the next. The protocol handles everything else.

Quick Reference Cheatsheet

QUICK REFERENCE

MCP Server Setup

Model Context Protocol · Local Development · macOS

TASK	COMMAND / ACTION
1 Check Python version	<code>python3 --version</code>
2 Install uv	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>
3 Find uv path	<code>which uv</code>
4 Create project	<code>mkdir my-server && cd my-server && uv init</code>
5 Install MCP SDK	<code>uv add mcp</code>
6 Test server	<code>uv run server.py</code>
7 Edit config	<code>nano ~/Library/Application\ Support/Claude/claude_desktop_config.json</code>
8 View logs	<code>cat ~/Library/Logs/Claude/mcp-server-*.log</code>

Read the companion story: [The Detective Who Knew Everyone — A Story About MCP → medium.com/@sakti.bagchi](https://medium.com/@sakti.bagchi)



10

Restart Claude Desktop

Cmd+Q → Reopen app

MCP · CLAUDE DESKTOP · macOS

docs.anthropic.com

By [Freedium.](#) — *made with care.*

[About](#)

[Privacy](#)

[Terms](#)

[RSS](#)

[GitHub](#)

[Codeberg](#)

